CultTrack AI

Smart Asset Management and Resource Allocation Platform

Updated Project Documentation

Project Scope
A full-stack web application for the Cultural Council of IIT Roorkee that centralizes inventory, booking requests, approvals, handovers, returns, audit logs, email notifications, password recovery, QR-assisted operations, analytics, and demand forecasting.

Field	Details
Submitted By	Yash Sharma (24116108), Arnav Saini (24116015)
Department	Electronics and Communication Engineering, IIT Roorkee
Repository	https://github.com/arnaviitr2135/smart-asset-management-system
Frontend	https://frontend-seven-lyart-83.vercel.app
Backend API	https://smart-asset-management-system-oiyh.onrender.com
Local Run	docker compose up --build
Last Updated	12 June 2026

1. Problem Understanding

1.1 Background and Context

The Cultural Council of IIT Roorkee manages a large pool of high-value shared assets used by sections, societies, and event teams. These assets include cameras, studio lights, recording equipment, sound systems, costumes, props, and event infrastructure. Because the same equipment moves between many students and events, the Council needs reliable visibility into who has an item, when it is due back, whether it is damaged, and whether it is available for a future event.

1.2 Manual Workflow Problems

- Separate spreadsheets maintained by different coordinators.
- Physical logbooks for check-ins and check-outs.
- Informal handovers coordinated through messages, calls, and word of mouth.
- No reliable real-time view of availability across upcoming dates.
- No permanent digital trail for accountability, condition, and return history.

1.3 Operational Pain Points

- Overbooking during major events because overlapping requests are hard to detect manually.
- Stock visibility gaps where an item appears available in one sheet but is physically checked out.
- Lost accountability for damaged or missing accessories.
- Slow issue and return operations during physical handovers.
- No analytics for utilization, shortage prediction, or purchase planning.

2. Proposed Solution

CultTrack AI replaces the fragmented manual system with a centralized portal. Members browse inventory and request resources, while admins use a control desk for approvals, issue, return, inventory management, health reports, notifications, analytics, and audit history.

Objective	Implementation
Prevent overbooking	Date-aware availability checks compare requested quantity against approved reservations and active loans without double-counting.
Improve accountability	Bookings, approvals, issues, returns, health reports, and admin actions are persisted and audited.
Speed operations	Admin desk, QR simulation, and focused check-in views reduce manual searching.
Improve communication	In-app and email notifications keep users/admins updated.
Support planning	Analytics and forecasting expose utilization and shortage risk.

3. Complete Feature Inventory

Area	Features	Users
Authentication	JWT login, registration, role-based sessions, protected routes.	All
Password Recovery	Forgot password, reset token, reset screen, privacy-safe response.	All
Email	Resend API, Gmail SMTP fallback, reset and workflow emails.	All
Inventory Catalog	Search, categories, shelf counts, active/maintenance/damaged states.	Members
Date Availability	Availability endpoint and modal count for selected dates.	Members
Booking Requests	Quantity/date/purpose validation and pending request queue.	Members/Admins
Approval Desk	Approve/reject requests and reserve physical stock.	Admins
Issue/Handover	Approved bookings become allocations with due dates and issuing admin.	Admins
Return Requests	Member return button, admin return request, return log, user confirmation.	All
Final Check-in	Condition on return, inventory restoration, damaged item maintenance.	Admins
Health Logs	Damage and maintenance reporting.	Admins
QR Simulation	QR payloads and scan simulation for fast lookup.	Admins
Notifications	Unread count, notification center, mark-read action.	All
Audit Trail	Permanent history of important actions.	Admins
Analytics	Metrics, utilization, active loans, overdue counts, forecast data.	Admins
Deployment	Docker, Vercel, Render, Neon.	Developers

4. User Roles and Workflows

4.1 Society Member Workflow

1. Register or log in.
2. Browse inventory by search/category.
3. Open an asset request modal and choose quantity, dates, and purpose.
4. Review Available for selected dates before submitting.
5. Submit booking and track status in Bookings and Loans.
6. Use Return Asset on issued loans or confirm admin return requests.
7. Receive in-app and email notifications.

4.2 Council Admin Workflow

1. Log in as admin.
2. Create/update inventory assets.
3. Review pending requests and approve/reject.
4. Issue approved assets during handover.
5. Monitor active loans and due dates.

6. Request users to return assets when needed.
7. Complete check-in with condition and notes.
8. Review health logs, audit logs, analytics, and forecasts.

5. System Architecture

Layer	Technology	Responsibility
Frontend	React, Vite, TypeScript, Tailwind, Recharts	UI for catalog, bookings, loans, admin desk, analytics, auth.
Backend	Node.js, Express, TypeScript	REST API, validation, RBAC, booking logic, notifications.
ORM	Prisma	Type-safe schema and PostgreSQL access.
Database	PostgreSQL	Users, assets, bookings, allocations, returns, logs, tokens.
Local Runtime	Docker Compose	Frontend, backend, and DB with seed data.
Production	Vercel, Render, Neon	Frontend hosting, backend Docker service, database.

6. Database Schema

Model	Purpose	Important Fields
User	Member and admin accounts.	email, passwordHash, fullName, role
Asset	Inventory items.	name, category, quantityAvailable, totalQuantity, status
Booking	Initial member request.	userId, assetId, quantity, dates, purpose, status
Allocation	Issued active loan.	bookingId, userId, issuedById, dueDate, status
Return	Completed return/check-in.	allocationId, receivedById, conditionOnReturn
ReturnRequest	Return coordination.	allocationId, source, status, notes
AssetHealthReport	Damage and maintenance.	assetId, reportedById, condition, notes
Notification	In-app alerts.	userId, title, message, isRead
AuditLog	Action history.	userId, action, details, createdAt
PasswordResetToken	Password reset security.	userId, tokenHash, expiresAt

7. API Overview

Route Group	Key Endpoints	Purpose
Auth	/register, /login, /forgot-password, /reset-password, /me	Accounts and sessions.
Assets	GET /assets, GET /:id, GET /:id/availability, POST/PUT/DELETE	Inventory and date availability.
Bookings	POST /bookings, GET /bookings, PUT /:id/approve, /reject	Requests and admin review.

Route Group	Key Endpoints	Purpose
Allocations	GET /allocations, POST /issue, POST /return	Issue and return check-in.
Return Requests	POST /request-return, GET /return-requests, POST /:id/respond	Return coordination.
Health	POST/GET /allocations/health	Damage and maintenance logs.
Analytics	GET /analytics, GET /analytics/forecast	Metrics and forecasting.
Audit	GET /audit	Admin action history.
Notifications	GET /notifications, PUT /:id/read	Notification center.
Health Check	GET /healthz	Backend/database status.

8. Booking and Availability Logic

- Catalog cards show current shelf stock, for example 4 / 8 in stock.
- The request modal calls /api/v1/assets/:id/availability for the selected dates.
- The backend prevents double-counting by treating approved bookings as the reservation source and not counting their issued allocations again.
- The request API validates quantity, date order, non-past start dates, date availability, and current shelf availability.
- Returns restore quantityAvailable and mark allocations as RETURNED.

9. Notifications, Email, and Password Reset

Capability	Behavior
In-app notifications	Unread count and notification feed for booking and return events.
Email provider	Resend API preferred; Gmail SMTP fallback supported.
Password reset	Reset token is created, hashed in DB, emailed, and expires.
Privacy	Forgot-password response does not reveal whether an email is registered.
Logs	Render logs show Resend, SMTP, skipped, or error states.

10. Analytics and Forecasting

The admin dashboard summarizes operational data and uses a lightweight weekly weighted moving-average style forecast to identify shortage risk without heavy machine-learning infrastructure.

- Dashboard metrics for assets, bookings, issued loans, overdue loans, and utilization.
- Forecast endpoint based on past booking/allocation activity.
- Shortage warnings when projected demand approaches available inventory.
- Recharts visualizations in the admin analytics UI.

11. Local Setup and Deployment

Docker Command
git clone https://github.com/arnaviitr2135/smart-asset-management-system.git
cd smart-asset-management-system
docker compose up --build

Service	URL	Notes
Frontend	http://localhost:5173	Vite React app.
Backend	http://localhost:5000	Express API.
Health	http://localhost:5000/healthz	API and DB check.
Production Frontend	https://frontend-seven-lyart-83.vercel.app	Vercel.
Production Backend	https://smart-asset-management-system-oiyh.onrender.com	Render.
Production DB	Neon PostgreSQL	Managed database.

12. Environment Variables

Variable	Used By	Purpose
DATABASE_URL	Backend	PostgreSQL/Neon connection string.
JWT_SECRET	Backend	JWT signing secret.
PORT	Backend	API port.
FRONTEND_URL	Backend	Reset links and allowed frontend origin.
RESEND_API_KEY	Backend	Email API key.
RESEND_FROM	Backend	Sender identity.
SMTP_USER / SMTP_PASS	Backend	Gmail fallback.
VITE_API_URL	Frontend	Render backend URL.

13. Security, Reliability, and Troubleshooting

- Passwords are stored as bcrypt hashes.
- JWT protects authenticated routes.
- Admin-only routes are guarded by role middleware.
- Password reset tokens are hashed and expire.
- Secrets are stored in environment variables, not committed.
- Audit logs preserve important operational actions.

Issue	Likely Cause	Resolution
Frontend cannot reach backend	Wrong VITE_API_URL or Render sleep/down.	Check /healthz and Vercel env vars.
Forgot password email missing	Email not registered or email provider env missing.	Check Render logs for Password Reset, Resend, SMTP, Email Error.
Wrong availability count	Backend not deployed to latest availability fix.	Deploy latest Render commit and use modal Available for selected dates.
Render Dockerfile error	Wrong Dockerfile/build context.	Use backend/Dockerfile.render and backend context.
Local stale data	Old Docker volume.	docker compose down -v, then docker compose up --build.

14. Future Enhancements

- Real QR scanner using device camera APIs.
- Calendar view for asset reservations.
- Exportable semester audit and budget reports.
- Verified production email domain for stronger deliverability.
- Fine-grained society/team ownership and approval policies.
- Automated overdue reminders and escalation workflows.

End of Updated Project Documentation